CSCI 432/532, Spring 2026

Homework 8

Due Monday, March 16 at 8:30am

Submission Requirements

- Type or clearly hand-write your solutions into a PDF format so that they are legible and professional. Submit your PDF on Canvas or Gradescope.
- Put each **sub**problem on a separate page and select the corresponding problem via the Gradescope interface.
- Do not submit your first draft. Type or clearly re-write your solutions for your final submission. If your submission is not legible, we will ask you to resubmit.

Collaboration and Resources

The *best* resources for completing the homework are (in no particular order):

- Your own notes from lectures and problem sessions, and/or lecture recordings.
- The lecture notes linked from the course website for the lecture day.
- The questions channel on Discord.
- Office hours.
- Discussions with classmates.

You may also access almost any resource in preparing your solution to the homework. The exception is that you may not seek answers to the problems on the homework directly, such as by pasting them into a GenAI tool, searching for solutions on a search engine, looking for them on Chegg or similar websites, or asking another person for the solution.

Additionally, you ***must***

- Write your solutions in your own words—this means that you should never be *copying* anything of substance from any source, and
- credit every resource you use, including GenAI tools, by providing links or full chat transcripts. If you use the provided LaTeX template, you can use the `Sources` environment for this. Ask if you need help!

> Remember, if you choose to use GenAI tools, not only must you provide a full transcript of your conversation (or a link to one), you must also use the following prompt (or some variation of it), and provide a full transcript of the conversation.

Prompt:

You are acting as a teaching assistant for an advanced undergraduate/graduate algorithms and computer science theory course.
Your role is not to provide full solutions or final answers.
Instead, you should act like we are having a conversation. Ask me a single question and let me respond. Avoid asking me many questions at once or giving me a lot of information at one time. Guide me using hints, leading questions, partial insights, and sanity checks. Help me debug my own proofs, algorithms, or reductions. Do not give a complete solution or full proofs. Be patient with me. Don’t give me details so that I can move on to the next part of a problem. Make sure that I understood before moving on.
The course uses Jeff Erickson’s Models of Computation lecture notes, so you should guide me to answer problems using the definitions, notation, and style from those notes.
Assume I am a serious student trying to learn, not to shortcut the assignment. I am attaching the assignment, so you should refuse to provide a complete solution to the problems listed. Of course, you can walk me through the "Solved Problems" listed at the end if I ask.

[Hint: Make sure you understand the definitions of the problems you are trying to prove NP-hard. Only a small amount of partial credit will be given for submissions that do not address the correct computational problems.]

Rubric for both of the following:

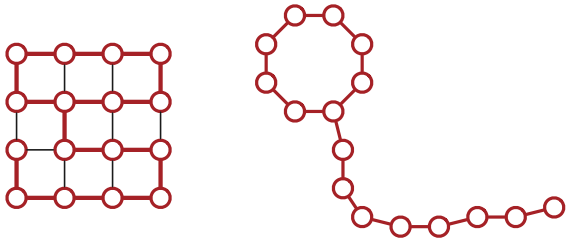
Standard NP-hardness rubric. 10 points =

+ 1 point for choosing a reasonable NP-hard problem X to reduce from.
  - The Cook-Levin theorem implies that *in principle* one can prove NP-hardness by reduction from *any* NP-complete problem. What we're looking for here is a problem where a simple and direct NP-hardness proof seems likely.
  - You can use any of the NP-hard problems listed in the lecture notes (except the one you are trying to prove NP-hard, of course).
+ 2 points for a *structurally sound* polynomial-time reduction. Specifically, the reduction must:
  - take an *arbitrary* instance of the declared problem X ***and nothing else*** as input,
  - transform that input into a corresponding instance of Y (the problem we're trying to prove NP-hard),
  - transform the output of the oracle for Y into a reasonable output for X, and
  - run in polynomial time.

  (The output transformation is usually trivial.) This is strictly about the structure of the reduction algorithm, not about its correctness. No credit for the rest of the problem if this is wrong.
+ 2 points for a *correct* polynomial-time reduction. That is, assuming a black-box algorithm that solves Y in polynomial time, the proposed reduction actually solves problem X in polynomial time.
+ 2 points for the "if" proof of correctness. (Every good instance of X is transformed into a good instance of Y.)
+ 2 points for the "only if" proof of correctness. (Every bad instance of X is transformed into a bad instance of Y.)
+ 1 point for writing "polynomial time"

- An incorrect but structurally sound polynomial-time reduction that still satisfies half of the correctness proof is worth at most 5/10 (= 1 for reasonable reduction source + 2 for structural soundness + 2 for half of the proof)
- A reduction in the wrong direction is worth at most 1/10 (for choosing a reasonable problem)

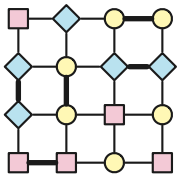
1. A *lollipop* of size ℓ is an undirected graph consisting of a (simple) cycle of length ℓ and a (simple) path of length ℓ, where one endpoint of the path lies on the cycle, and otherwise the cycle and the path are disjoint. Every lollipop of size ℓ has exactly 2ℓ vertices and 2ℓ edges. For example, the 4×4 grid graph shown below contains a lollipop subgraph of size 8.



Prove that it is NP-hard to find the size of the the largest lollipop subgraph of a given undirected graph.

2. Recall that a *3-coloring* of a graph assigns each vertex one of three colors, say red, yellow, and blue. A 3-coloring is *proper* if every edge has endpoints with different colors. The 3COLOR problem asks, given an arbitrary undirected graph G, whether G has a proper 3-coloring.

   Call a 3-coloring of a graph G *almost 3-coloring* if each vertex has *at most one neighbor* with the same color. The ALMOST3COLOR problem asks, given an arbitrary undirected graph G, whether G has a almost 3-coloring.



   (a) Consider the following attempt to prove that ALMOST3COLOR is NP-hard, using a reduction from 3COLOR.

> **Solution:** We reduce from 3COLOR. Given an arbitrary input graph G, we construct a new graph H by attaching a clique of 4 vertices to every vertex of G. Specifically, for each vertex v in G, the graph H contains three new vertices v_1, v_2, v_3, along with edges $vv_1, vv_2, vv_3, v_1v_2, v_1v_3, v_2v_3$. I claim that
>
> | G has a proper 3-coloring
if and only if
H has an almost 3-coloring. |
> |---|
>
> $\Longrightarrow$ Suppose G has a proper 3-coloring, using the colors red, yellow, and blue. Extend this color assignment to the vertices of H by coloring each vertex v_1 red, each vertex v_2 yellow, and each vertex v_3 blue. With this assignment, each vertex of H has at most one neighbor with the same color. Specifically, each vertex of G has the same color as one of the vertices in its gadget, and the other two vertices in v's gadget have no neighbors with the same color.
>
> $\not\Longleftarrow$ Now suppose H has an almost 3-coloring. Then G must have a proper 3-coloring because... um...
>
> ■

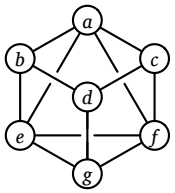
   Describe a graph G that does not have a proper 3-coloring, such that the graph H constructed by this reduction does have an almost 3-coloring.

   (b) Describe a small graph X with the following property: In every almost 3-coloring of X, every vertex of X has *exactly* one neighbor with the same color. [Hint: make sure you understand what this problem is asking for. You need to ask yourself: for *every possible* almost 3-coloring of X, does every vertex have exactly one neighbor with the same color? It is not sufficient for *some* almost 3-coloring of X to have this property.]

(c) Describe a correct polynomial-time reduction from 3COLOR to ALMOST3COLOR. *[Hint: Use your graph from part (b) as a gadget.]* This reduction will prove that ALMOST3COLOR is indeed NP-hard.

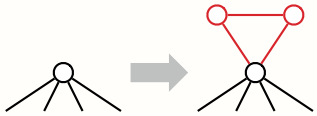
Solved Problem

3. A *double-Hamiltonian tour* in an undirected graph G is a closed walk that visits every vertex in G exactly twice. Prove that it is NP-hard to decide whether a given graph G has a double-Hamiltonian tour.



This graph contains the double-Hamiltonian tour $a \to b \to d \to g \to e \to b \to d \to c \to f \to a \to c \to f \to g \to e \to a$.

Solution: We prove the problem is NP-hard with a reduction from the standard Hamiltonian cycle problem. Let G be an arbitrary undirected graph. We construct a new graph H by attaching a small gadget to every vertex of G. Specifically, for each vertex v, we add two vertices $v^\sharp$ and $v^\flat$, along with three edges $vv^\flat$, $vv^\sharp$, and $v^\flat v^\sharp$.



A vertex in G and the corresponding vertex gadget in H.

Now I claim that

G has a Hamiltonian cycle
if and only if
H has a double-Hamiltonian tour.

$\Longrightarrow$ Suppose G contains a Hamiltonian cycle $C = v_1 \to v_2 \to \cdots \to v_n \to v_1$. We can construct a double-Hamiltonian tour of H by replacing each vertex v_i in C with the following walk:

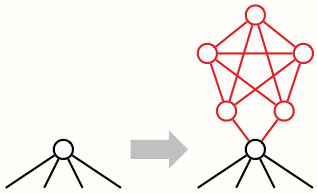
$$\cdots \to v_i \to v_i^\flat \to v_i^\sharp \to v_i^\flat \to v_i^\sharp \to v_i \to \cdots$$

$\Longleftarrow$ Conversely, suppose H has a double-Hamiltonian tour D. Consider any vertex v in the original graph G; the tour D must visit v exactly twice. Those two visits split D into two closed walks, each of which visits v exactly once. Any walk from $v^\flat$ or $v^\sharp$ to any other vertex in H must pass through v. Thus, one of the two closed walks visits only the vertices v, $v^\flat$, and $v^\sharp$. Thus, if we remove the vertices and edges in $H \setminus G$ from D, we obtain a closed walk in G that visits every vertex in G exactly once.

Given any graph G, we can clearly construct the corresponding graph H in polynomial time by brute force.

With more effort, we can construct a graph H that contains a double-Hamiltonian tour ***that traverses each edge of H at most once*** if and only if G contains a Hamiltonian

cycle. For each vertex v in G we attach a more complex gadget containing five vertices and eleven edges, as shown on the next page.

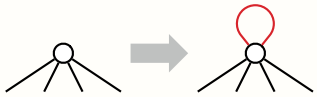


A vertex in G, and the corresponding modified vertex gadget in H.

■

Rubric: 10 points, standard polynomial-time reduction rubric. This is not the only correct solution.

Solution (self-loops): We attempt to prove the problem is NP-hard with a reduction from the Hamiltonian cycle problem. Let G be an arbitrary undirected graph. We construct a new graph H by attaching a self-loop every vertex of G. Given any graph G, we can clearly construct the corresponding graph H in polynomial time.



An incorrect vertex gadget.

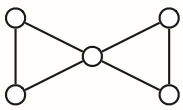
Now I claim that

G has a Hamiltonian cycle if and only if H has a double-Hamiltonian tour.

$\Longrightarrow$ Suppose G has a Hamiltonian cycle $v_1 \to v_2 \to \cdots \to v_n \to v_1$. We can construct a double-Hamiltonian tour of H by alternating between edges of the Hamiltonian cycle and self-loops: $v_1 \to v_1 \to v_2 \to v_2 \to v_3 \to \cdots \to v_n \to v_n \to v_1$.

$\not\Longleftarrow$ Um...

Unfortunately, if H has a double-Hamiltonian tour, we *cannot* conclude that G has a Hamiltonian cycle, because we cannot guarantee that a double-Hamiltonian tour in H uses *any* self-loops. The graph G shown below is a counterexample; it has a double-Hamiltonian tour (even before adding self-loops!) but no Hamiltonian cycle.



This graph has a double-Hamiltonian tour.

■

Some useful NP-hard problems. You are welcome to use any of these in your own NP-hardness proofs, except of course for the specific problem you are trying to prove NP-hard.

CircuitSat: Given a boolean circuit, are there any input values that make the circuit output True?

3Sat: Given a boolean formula in conjunctive normal form, with exactly three distinct literals per clause, does the formula have a satisfying assignment?

MaxIndependentSet: Given an undirected graph G, what is the size of the largest subset of vertices in G that have no edges among them?

MaxClique: Given an undirected graph G, what is the size of the largest complete subgraph of G?

MinVertexCover: Given an undirected graph G, what is the size of the smallest subset of vertices that touch every edge in G?

MinSetCover: Given a collection of subsets $S_1, S_2, \ldots, S_m$ of a set S, what is the size of the smallest subcollection whose union is S?

MinHittingSet: Given a collection of subsets $S_1, S_2, \ldots, S_m$ of a set S, what is the size of the smallest subset of S that intersects every subset S_i?

3Color: Given an undirected graph G, can its vertices be colored with three colors, so that every edge touches vertices with two different colors?

ChromaticNumber: Given an undirected graph G, what is the minimum number of colors required to color its vertices, so that every edge touches vertices with two different colors?

HamiltonianPath: Given graph G (either directed or undirected), is there a path in G that visits every vertex exactly once?

HamiltonianCycle: Given a graph G (either directed or undirected), is there a cycle in G that visits every vertex exactly once?

TravelingSalesman: Given a graph G (either directed or undirected) with weighted edges, what is the minimum total weight of any Hamiltonian path/cycle in G?

LongestPath: Given a graph G (either directed or undirected, possibly with weighted edges), what is the length of the longest simple path in G?

SteinerTree: Given an undirected graph G with some of the vertices marked, what is the minimum number of edges in a subtree of G that contains every marked vertex?

SubsetSum: Given a set X of positive integers and an integer k, does X have a subset whose elements sum to k?

Partition: Given a set X of positive integers, can X be partitioned into two subsets with the same sum?

3Partition: Given a set X of $3n$ positive integers, can X be partitioned into n three-element subsets, all with the same sum?

IntegerLinearProgramming: Given a matrix $A \in \mathbb{Z}^{n \times d}$ and two vectors $b \in \mathbb{Z}^n$ and $c \in Z^d$, compute $\max\{c \cdot x \mid Ax \leq b, x \geq 0, x \in \mathbb{Z}^d\}$.

FeasibleILP: Given a matrix $A \in \mathbb{Z}^{n \times d}$ and a vector $b \in \mathbb{Z}^n$, determine whether the set of feasible integer points $\max\{x \in \mathbb{Z}^d \mid Ax \leq b, x \geq 0\}$ is empty.

Draughts: Given an $n \times n$ international draughts configuration, what is the largest number of pieces that can (and therefore must) be captured in a single move?

SuperMarioBrothers: Given an $n \times n$ Super Mario Brothers level, can Mario reach the castle?